CS23702 – ARTIFICIAL INTELLIGENCE

AI-BASED YOTE GAME

USING ADVERSARIAL SEARCH ALGORITHMS

COMPARATIVE PERFORMANCE ANALYSIS OF ALPHA-BETA, NEGASCOUT AND MTD-f

TEAM NO : 18

TEAM MEMBERS

1. Joshika Shree V - 2023103043
2. Santhosh K - 2023103552
3. Lavanya U - 2023103558

1. OBJECTIVE OF THE PROJECT

The main objective of this project is to develop an AI-based Yote board game that implements and compares different adversarial search algorithms for intelligent game decision-making.

- To develop a digital implementation of the Yote board game.
- To implement Alpha-Beta Pruning, NegaScout, and MTD-f adversarial search algorithms.
- To develop a heuristic evaluation function for evaluating Yote board positions.
- To implement search enhancements such as Move Ordering, Transposition Table, and Hash Move.
- To evaluate the algorithms using metrics such as execution time, nodes explored, search depth, and branching factor.
- To compare the performance of different algorithms and enhancement configurations.
- To identify the best-performing search algorithm/configuration for the implemented Yote game.

2. PROBLEM STATEMENT

Traditional game-playing programs need to examine a large number of possible future game states before selecting an appropriate move. As the number of possible moves increases, exhaustive search becomes computationally expensive.

Yote is a two-player, deterministic, zero-sum board game played on a 6×5 board. Each player starts with 12 pieces, and pieces can be moved horizontally or vertically. Capturing an opponent's piece is also possible according to the game rules.

A basic MiniMax-based search can become expensive because its search complexity grows exponentially with the branching factor and search depth. The reference paper estimates an average branching factor of approximately 36 for Yote.

Therefore, there is a need to investigate efficient adversarial search techniques that can explore the game tree intelligently while reducing unnecessary computation.

This project addresses this problem by implementing Alpha-Beta, NegaScout, and MTD-f and comparing their performance under different search-enhancement configurations.

3. ABSTRACT

Artificial Intelligence techniques have been widely used in board games to enable machines to make intelligent decisions against human or computer opponents. This project presents an AI-based implementation of the Yote board game using adversarial search algorithms.

The proposed system implements three search algorithms: Alpha-Beta Pruning, NegaScout, and MTD-f. A heuristic evaluation function is used to estimate the quality of non-terminal game states during depth-limited search. To improve search efficiency, the system incorporates Move Ordering, Transposition Tables, and the Hash Move heuristic.

The system provides different configurations of the search algorithms and evaluates their performance on Yote game positions. Performance is measured using parameters such as execution time, number of nodes explored, search depth, and effective branching factor.

The experimental results are used to determine which adversarial search algorithm and enhancement configuration provides better performance for the implemented Yote game. The project therefore demonstrates the application of adversarial search, heuristic evaluation, and search optimization techniques in game-playing Artificial Intelligence.

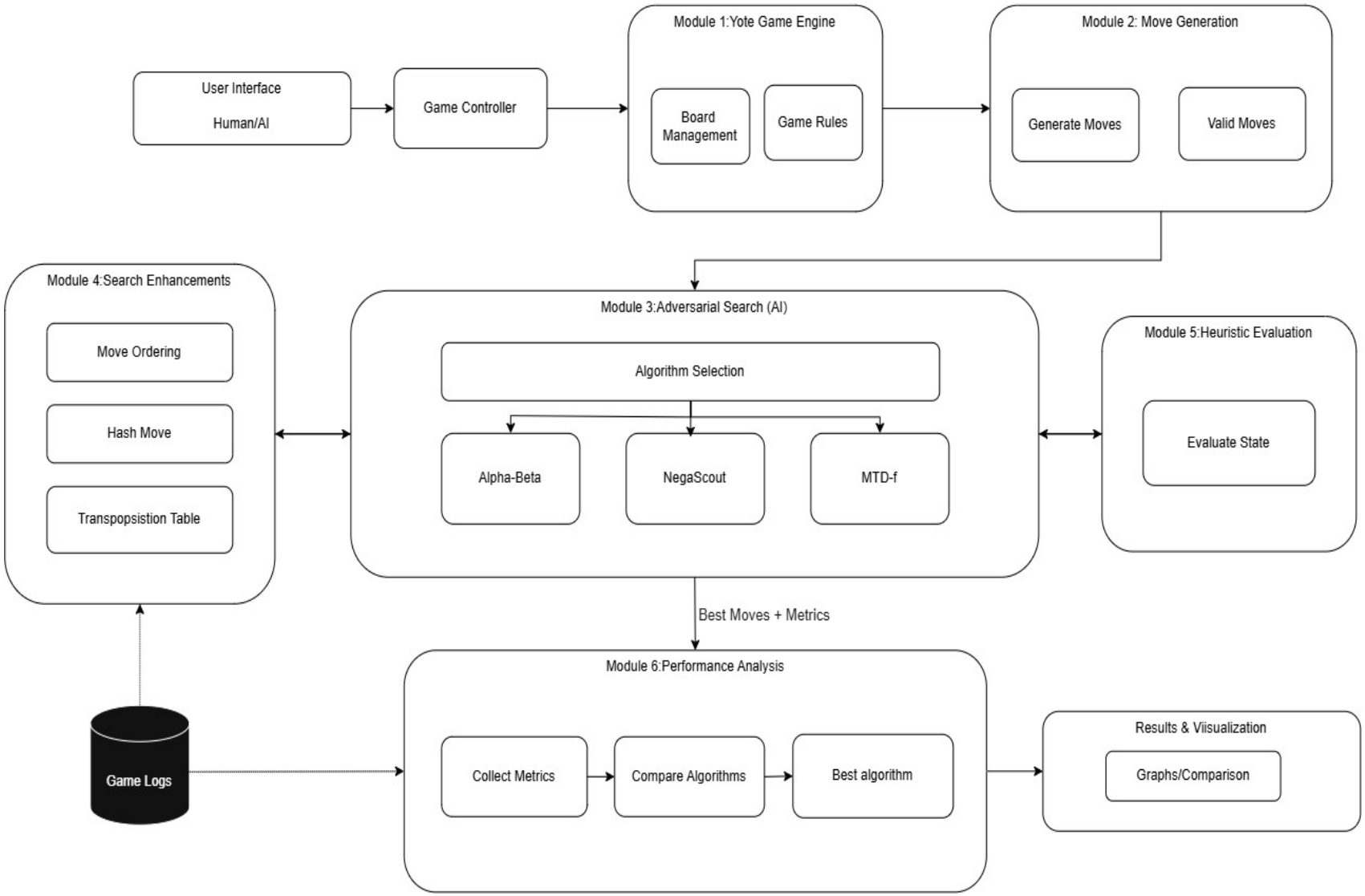
4. OVERALL ARCHITECTURE

The proposed system consists of six major modules:

- Yote Game & Board Management
- Move Generation & State Management
- Adversarial Search
- Search Enhancements
- Heuristic Evaluation
- Performance Analysis & Comparison

The system accepts a game state from the Yote game engine and generates legal moves. The adversarial search module uses the selected algorithm to explore possible future states. Search enhancements such as Move Ordering, Transposition Table and Hash Move to improve search efficiency. The heuristic evaluation module provides scores for game states, while the performance analysis module records and compares the search results.

OVERALL SYSTEM ARCHITECTURE



5. DETAILED DESIGN – MODULE-WISE

Module 1 – Yote Game & Board Management

This module is responsible for implementing the basic Yote game environment.

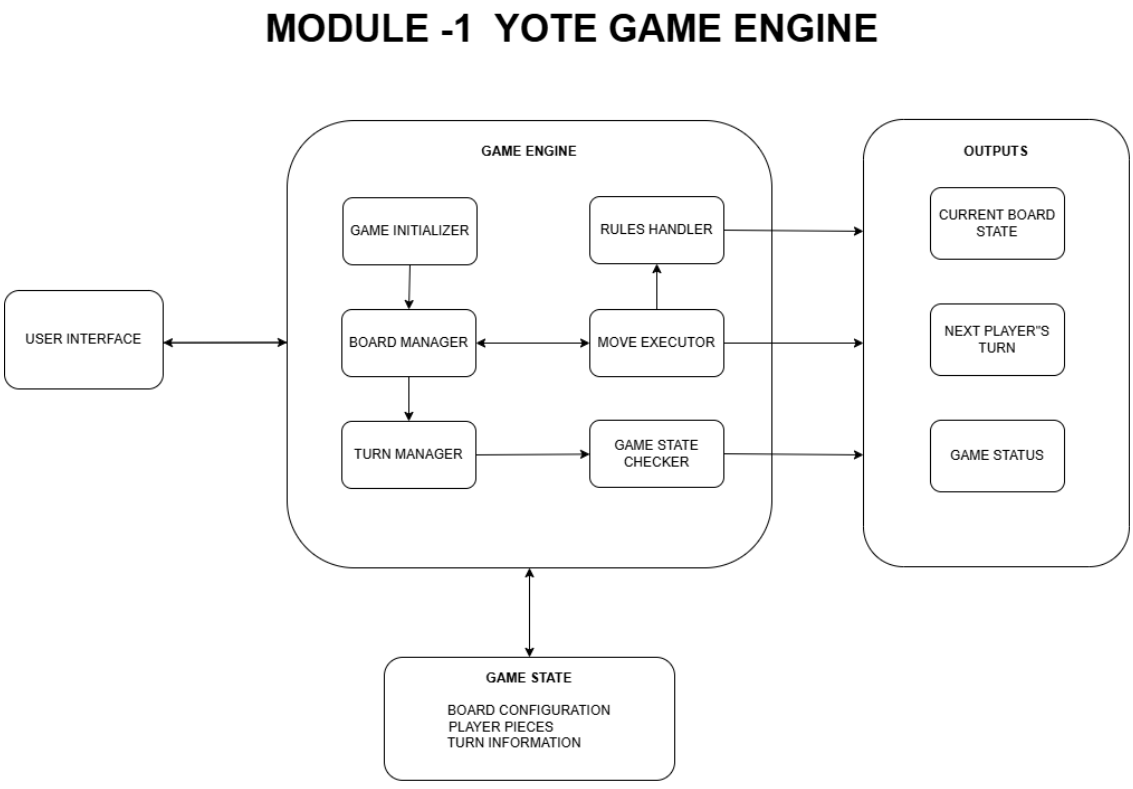
FUNCTIONS:

- Board initialization
- 6×5 board representation
- Piece placement
- Player turn management
- Move execution
- Capture handling
- Game-state maintenance
- Win/loss condition checking

The module receives player or AI commands and updates the board accordingly.

Input: Player move / AI move

Output: Updated board state and game status



Module 2 – Move Generation & State Management

This module generates all possible legal moves from a given Yote board state.

The module first identifies the current player's pieces and examines their possible movements. Normal movements and capture movements are generated separately and then validated according to the game rules.

FUNCTIONS:

- Identify current player's pieces
- Generate normal moves
- Generate capture moves
- Validate moves
- Apply moves
- Generate successor states
- Maintain valid-move list

The resulting legal moves and successor states are passed to the adversarial search module.

Input: Current board state

Output: Valid moves and successor states

Module 3 – Adversarial Search

This is the main AI decision-making module.

The module contains the three search algorithms:

Alpha-Beta Pruning

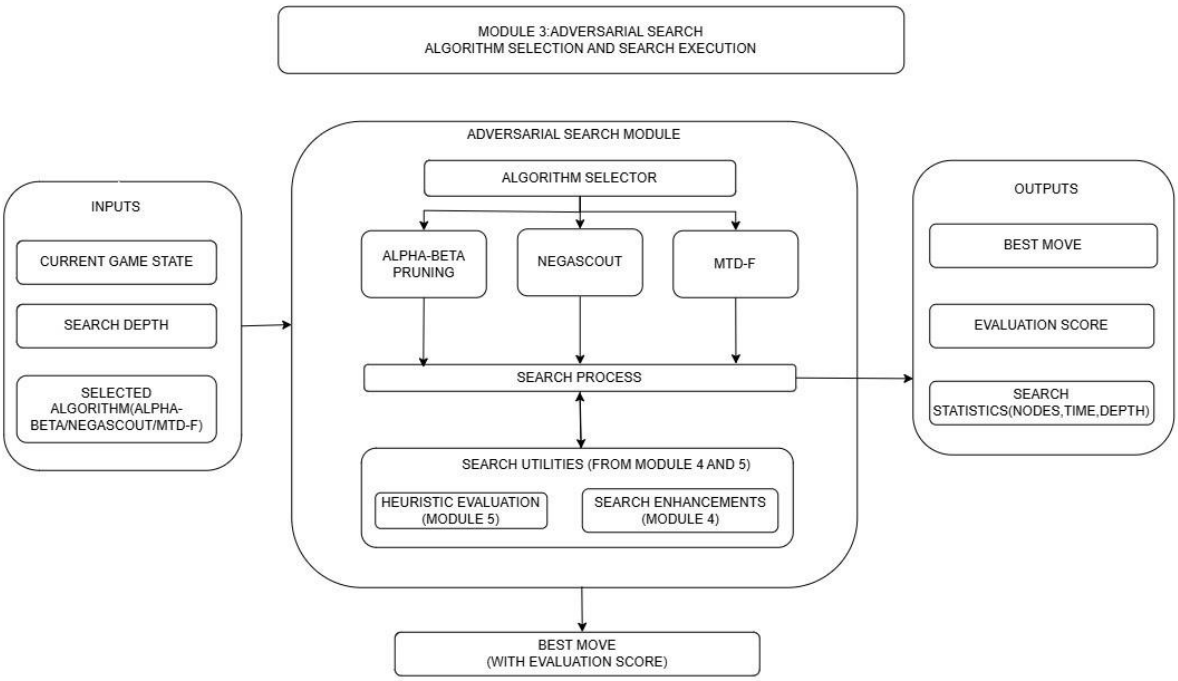
Alpha-Beta reduces the search space by pruning branches that cannot influence the final decision. With good move ordering, Alpha-Beta can significantly reduce the effective search space compared with basic MiniMax.

NegaScout

NegaScout uses a null-window search after examining the first successor. If the initial search fails to establish the expected bound, a re-search is performed. Its performance is highly dependent on move ordering.

MTD-f

MTD-f performs repeated null-window searches using Alpha-Beta with memory. It can benefit significantly from a transposition table.



Module 4 – Search Enhancements

This module contains techniques used to improve search efficiency. **Move Ordering**

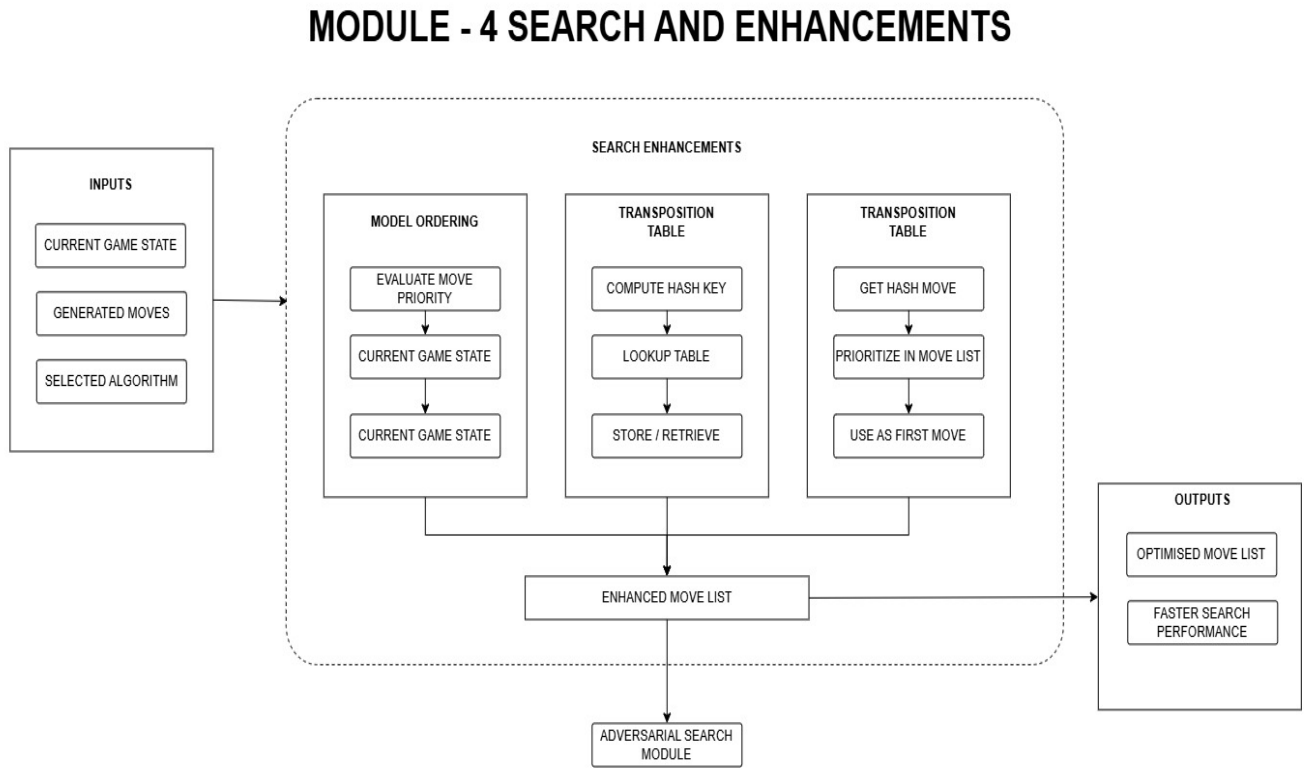
Moves are arranged so that promising moves are searched first. Good ordering can increase pruning efficiency, particularly for Alpha-Beta and NegaScout.

Transposition Table

Previously evaluated game states are stored and reused when the same board position is encountered again.

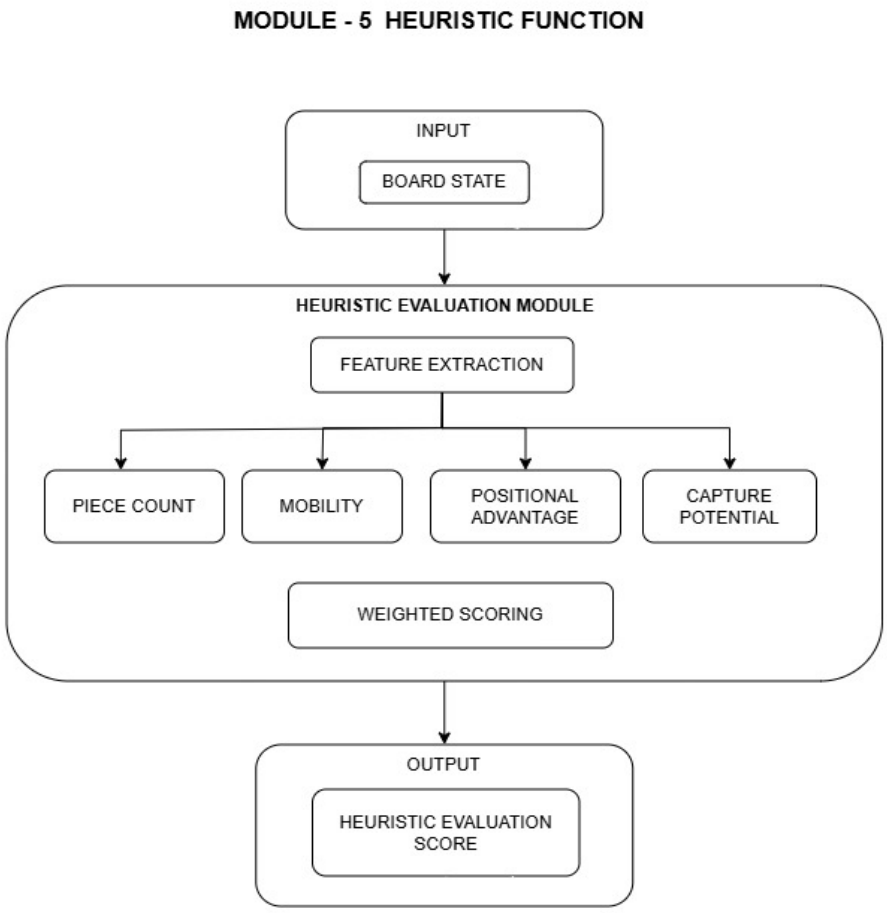
Hash Move

The best move stored for a previously encountered state is retrieved and considered early during subsequent searches.



Module 5 – Heuristic Evaluation

The search algorithms cannot always continue until the actual end of the game because the search tree can become very large. Therefore, a heuristic evaluation function is used to estimate the quality of a board position at the search depth limit.



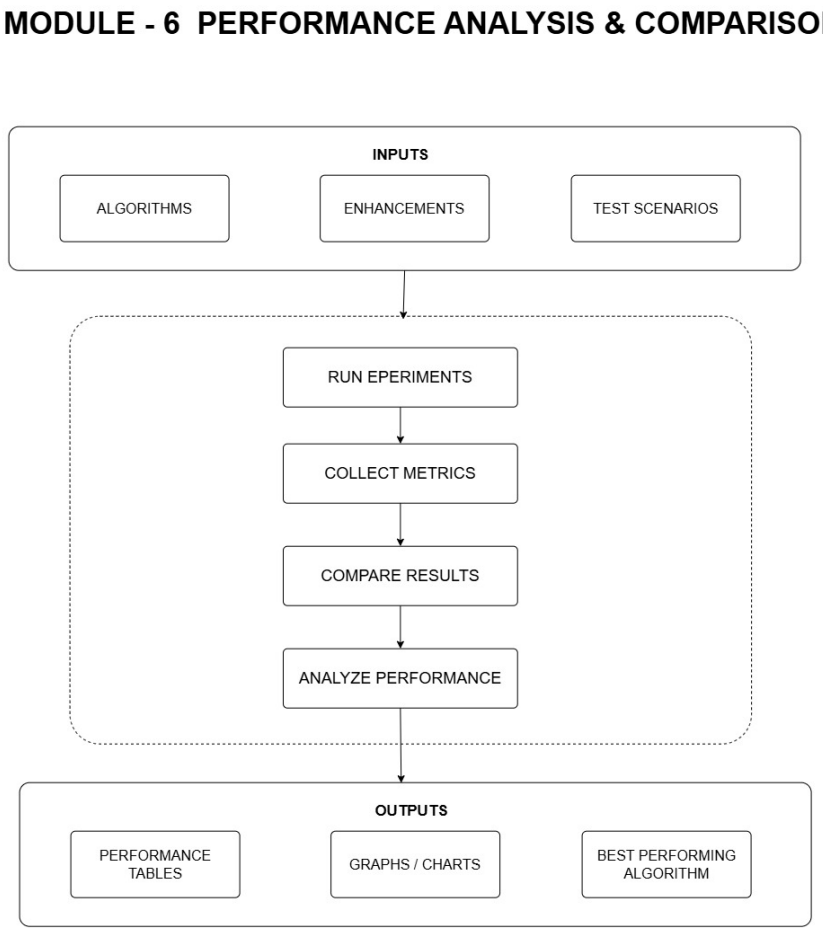
Module 6 – Performance Analysis & Comparison

This module evaluates the performance of the implemented algorithms.

For each algorithm and configuration, the system records relevant search statistics.

Performance Metrics

- Execution Time
- Measures search speed
- Nodes Expanded
- Search Depth
- Branching Factor
- Win/Loss/Draw



6. EXPECTED OUTPUT

The system should produce:

1. Best Move

Selected Algorithm: MTD-f

Search Depth: 6

Best Move: (x1,y1) → (x2,y2)

2. Performance Statistics

Execution Time

Nodes Expanded

Search Depth

Branching Factor

3.Comparison

Compare all the three algorithms Alpha-Beta, NegaScout, MTD-F and choose the best

7. PROPOSED AI TECHNIQUES

The project uses the following AI techniques:

- Adversarial Search
- MiniMax / NegaMax concepts
- Alpha-Beta Pruning
- NegaScout / Principal Variation Search
- MTD-f
- Heuristic Search
- Move Ordering
- Transposition Tables
- Hash Move Heuristic
- Depth-Limited Search
- Game-State Evaluation

8. TECHNOLOGY STACK

Component	Technology
Programming Language	**Python**
Game Interface	**Pygame**
AI Algorithms	Custom Python implementation
Data Processing	**Pandas**
Visualization	**Matplotlib**
Data Storage	CSV / JSON
Testing	Python

9. CONCLUSION

The proposed project develops an AI-based Yote game using adversarial search techniques and provides a comparative evaluation of Alpha-Beta, NegaScout, and MTD-f. By incorporating search enhancements such as Move Ordering, Transposition Tables, and Hash Move, the system investigates how different optimization techniques affect the efficiency of game-tree search.

The performance analyzer provides quantitative comparisons based on execution time, nodes explored, search depth, and branching factor. The experimental results can then be used to identify the most efficient algorithm and configuration for the implemented Yote environment.

The reference study found that **MTD-f with Transposition Table, Hash Move and Move Ordering achieved the best overall performance** in its experiments, while also observing that the relative performance depends on the game stage and search configuration.